Projet développement logiciel :

Architecture globale du projet

ETAPES IMPORTANTES DU PROJET :

1) Prétraitement des données

Objectif : prépare les images pour qu'elles soient toutes dans un format exploitable par le modèle VAE (qui a besoin d'entrées de même taille, même type de valeur, même format,…)

Fonctions :

- Lire les images depuis le dossier de données
- Vérifier que les images sont bien valides/exploitables (normalement oui),
- Voir si elles sont toutes de la même taille, sinon les convertir toutes à la même taille
- Vérifier si elles ont toutes le même type (RGB), sinon conversion au même type
- Normaliser les pixels ?
- Faire en sorte qu'elles soient toutes cadrées de la même façon ? (centrage visage, recadrage,ect…)

2) Modèle Auto encodeur : VAE

Objectif : Construire le VAE qui prend en entrée une image, la compresse dans un espace latent puis la reconstruit.

Fonctions :

- Encodeur convolutionnel : couches de convolutions, activations, pooling/strides éventuels (voir cours du tuteur)
- Projection vers espace latent : calcule de moyenne et variance/log-variance, choisir la dimension de l'espace latent
- Décodeur convolutionnel : couches de déconvolution, reconstruction image finale (attention si on a normalisé les images avant)
- Penser à sauvegarder les images décodées par décodeur

3) Entrainement du modèle

Objectif : Faire apprendre le VAE à reconstruire correctement les images tout en ayant un espace latent qui soit exploitable (pour modifications génétiques)

Fonctions :

- Split les images en données training + set
- Train le modèle
- Calcul de fonction de coût du VAE : erreur de reconstruction + terme de régularisation KL
- Mise à jour des poids du réseau
- Suivre évolution de l'apprentissage (graphes, courbes de loss,…), éviter l'overfitting et garder les meilleurs poids

Remarque : tuteur avait dit de prendre un jeu de donnée plus petit pour faire l'entrainement et il avait donné un je crois (lui demander si jamais)

Autre remarque : s'assurer que l'ordi qui fait l'entrainement possède un GPU sinon l'opération peut prendre beaucoup de temps (GPU sert à faire des opérations en parallèle sur plusieurs milliers de cœurs alors que CPU ne possède pas beaucoup de cœur donc prend plus de temps à faire les calculs, ici vu qu'on fait une VAE : bcp d'opérations de matrices,…)

4) Evaluation du modèle :

Objectif : Mesurer la qualité du modèle entrainé, vérifier qu'il reconstruit bien les images et que l'espace latent est exploitable

Fonctions :

- Evaluer la performance sur les images non utilisées lors du training
- Idée : visualisation de l'espace latent, visualisions d'une comparaison d'images original/reconstruction

5) Exploration dans l'espace latent

Objectif : utiliser les vecteurs latents pour faire des manipulations sur les images. Chaque image est transformée en un vecteur, par exemple :

- image A -> vecteur z_A
- image B -> vecteur z_B

5.1) Interpolation entre 2 images :
Rôle : passer progressivement d'une image A vers une image B
- $z(t)=(1-t).z_A + t.z_B$ avec t entre 0 et 1
- Génération de plusieurs valeurs de t entre 0 et 1
- Puis décoder les différentes images interpolées

5.2) Mélange génétique : 'croisement' entre 2 images
Rôle : créer une nouvelle image hybride en combinant deux images dans l'espace latent
$z = \alpha. z_A + (1-\alpha). z_B$ certaines caractéristiques viennent plus de chez A d'autres de chez B
- Génération d'une image

5.3) Modification d'attributs :
Rôle : on peut essayer de repérer des directions associées à un trait dans le vecteur latent (par exemple cheveux plus présents, visage plus allongé). Pour pouvoir repérer ces directions, print les vecteurs latents de certaines images (par exemple : on print plusieurs vecteurs d'images avec lunettes et on regarde quelles dimensions dans le vecteur sont communes).
Si on arrive à faire ça on peut créer une direction d dans l'espace latent et l'appliquer aux images :

$z'=z + \lambda d$ puis décoder z' (λ : intensité du changement)

Remarque : partie un peu plus compliquée à faire, pas garanti d'avoir de bons résultats.

6) Interface graphique :

Objectif : app où l'utilisateur peut utiliser le projet sans toucher au code.

7) Déploiement et installation

Objectif : rendre projet réutilisable sur un autre ordinateur.

On utilise Docker qui permet d'éviter des problèmes du types : mauvaise version de Python, package manquant, incompatibilité de bibliothèque,… Docker permet de créer un environnement avec : les bonnes versions de Python, des bibliothèques, …

Architecture (idée) :

Projet_dev_autoportrait/

|

|--- README.md (explique le projet, cmt l'installer, cmt le lancer)

|--- requirement.txt (liste des bibliothèques utilisés dans le projet -> utile pour créer l'environnement)

|--- Dockerfile (pour lancer le projet sur un autre ordi et éviter les problèmes de versions)

|--- main.py (on peut faire un code principal ici)

|

|--- data/

| |--- raw_data/ (images d'origine brutes)

| |--- processed_data/ (pour éviter de refaire le preprocessing à chaque fois, les garder)

| --- checkpoints/ (contient les poids du modèle VAE sauvegardés pdt entrainement, comme ça application pourra directement reprendre ces poids pour générer les images)

|

|---outputs/ (résultats générés, : reconstruction, interpolation, mélanges,…)

|--- source/ (code interne, juste les fonctions)

|--- preprocessing.py (ce qui prépare les images : lecture, conversions,..)

|--- dataset.py (

|--- vae_model.py (contient encodeur, décodeur, loss,..)

|--- train.py (contient les boucles d'entraînement et de validation)

|--- latent_space.py (les opérations dans l'espace latent : mélanges,…)

|--- app.py (interface graphique)

|--- utils.py (les fonctions qui reviennent un peu partout, pour éviter des mettre partout)

|--- scripts/ (appelle les fonctions de src et fait les grandes étapes)

|--- preprocess_data.py (lance le preprocessing)

|--- train_model.py (lance entrainement)

|--- launching_app.py (lance application)